

Lab 2 - Cloud Data, Stat 214, Spring 2025

March 20, 2026

1 Introduction

2 EDA

3 Feature Engineering

4 Transfer Learning

4.1 Motivation

In this project, we only have a small amount of labeled data, but we have many more unlabeled data. It is hard to train a supervised model well, since the model may easily overfit and may not learn stable patterns from such a small target set. To deal with this problem, we use transfer learning with an autoencoder.

Our basic idea is simple. First, we train an autoencoder on all unlabeled data, so that it can learn useful local patterns from reconstruction. After that, we fine-tune the pretrained model on the three labeled target images. But in this fine-tuning step, we still do not use the expert labels, the labels are kept for the later classification task. This lets the encoder adjust to the target images before we use it for downstream prediction.

This strategy is helpful for two reasons. First, pretraining gives the model a better starting point than random initialization. Second, fine-tuning makes the learned representation closer to the target. Because of this, the latent features from the encoder should be more useful for the classification part later.

4.2 Part A: Transfer Learning with Autoencoder

4.2.1 Overview of Our Pipeline

Our pipeline has two main stages. In the first stage, we pretrain an autoencoder on a large set of unlabeled images. In the second stage, we fine-tune that pretrained model on the three labeled target images, still using reconstruction as the training objective. Inside the second stage, we first do cross-validation fine-tuning to choose the best hyperparameters: learning rate and the number of epochs, and then we do one final fine-tuning run on all three images.

The output of this part is not a classifier. Instead, it is a fine-tuned encoder that can turn each spatial patch into a latent vector. These latent vectors are then used as inputs for the classification models.

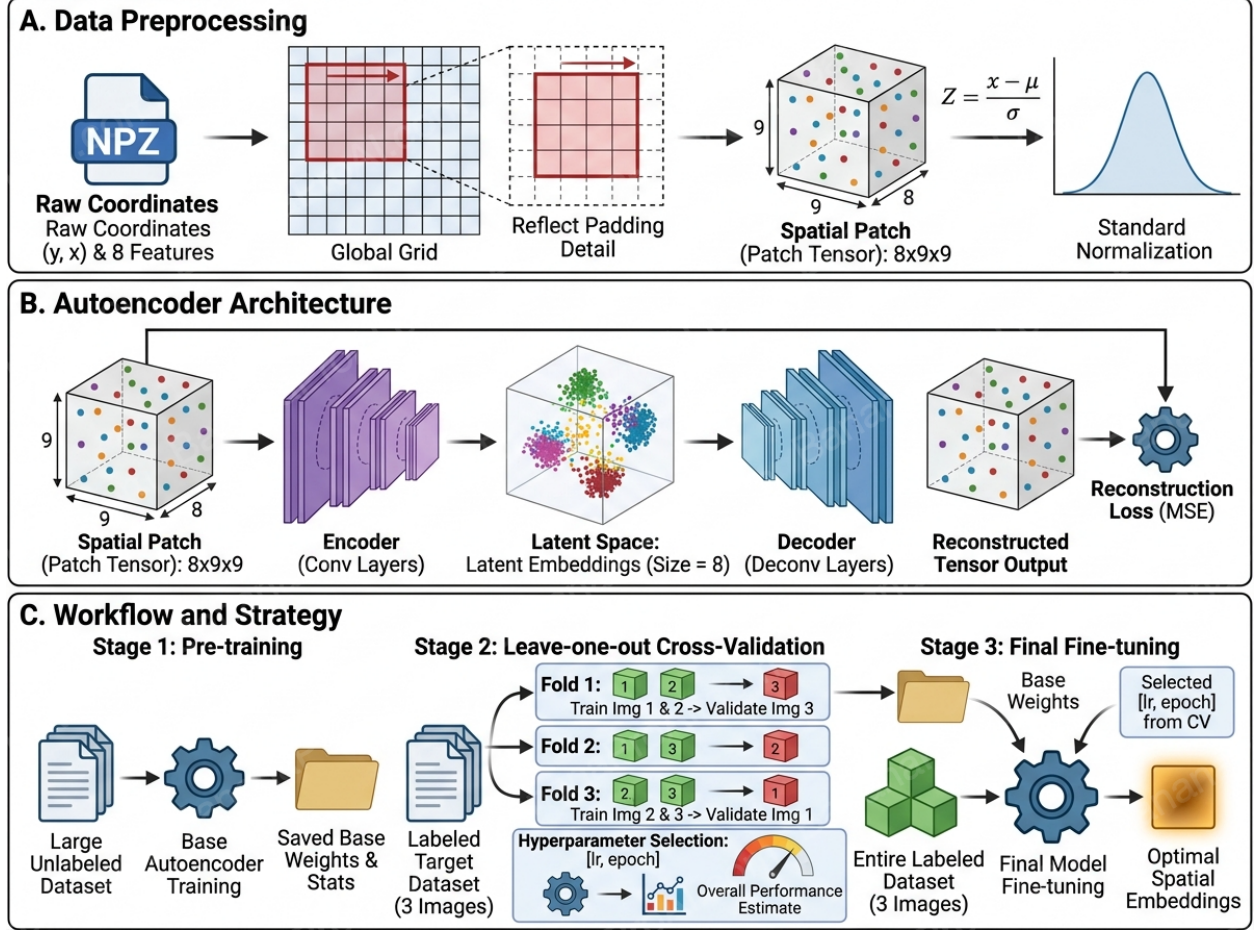


Figure 1: Overview of our transfer learning pipeline. Panel A shows data preprocessing: raw coordinate-based features are reorganized into dense spatial tensors, local 9×9 patches are extracted with reflect padding, and channel-wise normalization is applied. Panel B shows the autoencoder, which maps each $8 \times 9 \times 9$ patch to a latent embedding and reconstructs the input patch using mean squared error loss. Panel C shows the full workflow: unlabeled pretraining, leave-one-image-out cross-validation fine-tuning, and final fine-tuning on all three labeled target images.

4.2.2 Data Preparation

Each .npz file is loaded as a pixel-level table. The first two columns are the spatial coordinates (y, x), and the remaining columns are feature values. For labeled files, there is one extra label column. During autoencoder training, this label column is removed, because the reconstruction task does not need it.

After loading the files, we compute one global coordinate range over all images. Then each image is rebuilt into a dense tensor of shape

$$(channels, height, width).$$

In our case, the number of channels is 8. Using one shared coordinate frame makes all images have the same tensor shape, which is convenient for patch extraction and batching.

We then normalize the data channel by channel. The mean and standard deviation are computed from the pretraining data and saved. Later, the same normalization statistics are reused in fine-tuning. We do this to keep the data scale consistent across the two stages.

For training, we extract a local patch around each valid pixel. The patch size is 9×9 , so each sample has

shape

$$8 \times 9 \times 9.$$

For boundary pixels, we use reflect padding. This patch-based input is important, because it lets the model see local neighborhood information instead of using only one pixel at a time.

4.2.3 Pretraining on Unlabeled Images

In the pretraining stage, we use 161 unlabeled images. The split is done at the image level: 80% of the images are used for training and 20% are used for validation. We choose image-level splitting because patches from the same image are strongly related. If we split at the patch level, training and validation would contain very similar patches, and the validation loss would look better than it really is.

After normalization and patch extraction, all training patches are combined into one training set, and all validation patches are combined into one validation set. The autoencoder is then trained to reconstruct the input patches. The best checkpoints are selected by validation loss, and the normalization statistics are also saved for later use.

In our experiments, the pretraining learning rate is 10^{-3} , the batch size is 1024, and the maximum number of epochs is 20. This stage gives the encoder a reasonable representation before we move to the small target dataset.

4.2.4 Fine-Tuning on the Three Labeled Images

After pretraining, we load the pretrained checkpoint and the saved normalization statistics. Then we fine-tune the autoencoder on the three labeled target images. Even though these images have expert labels, the fine-tuning loss is still only reconstruction loss. The purpose here is not to train a classifier yet, but to make the pretrained representation fit the target images better.

In our final setup, all model parameters are trainable during fine-tuning. We do not freeze the encoder or decoder. Since the target images are the final domain we care about, we let the full model adjust to them.

We use two fine-tuning steps.

Cross-validation fine-tuning. First, we do leave-one-image-out cross-validation on the three labeled images. In each fold, two images are used for training and one image is used for validation. This gives three folds in total. Since the dataset is very small, this step is useful for checking whether the model behaves similarly across different target images. For each fold, we record the best validation loss, and we also report the mean and standard deviation over the three folds. We choose the hyperparameters learning rate and the number of epochs based on the validation loss.

Final fine-tuning. After cross-validation, we train again on all three labeled images together. This run is used to produce the final checkpoint for embedding extraction. At this point, we are no longer trying to compare folds or choose settings. We just want to use all target images to get the final encoder.

For fine-tuning, we use a smaller learning rate, 10^{-4} . The maximum number of epochs is 20 for the cross-validation runs and 5 for the final fine-tuning run. Since the model already starts from pretrained weights, a smaller learning rate is enough for adaptation.

4.2.5 Outputs of This Part

This part produces several outputs:

- pretrained checkpoints,
- three cross-validation fine-tuning checkpoints,
- final fine-tuning checkpoints.

These outputs are used later to extract latent vectors from the encoder for the classification models in the next modeling part. Overall, this part gives us a representation model that first learns from many unlabeled images and then adjusts to the small target dataset before downstream supervised learning.

Model	Activation	Latent dim	Selected checkpoint / notes
Baseline	ReLU	8	Final embeddings were exported from the baseline <code>finetune_final_004</code> run, initialized from <code>pretrain-epoch=014-val_loss=0.1063.ckpt</code> .
Modified	LeakyReLU(0.1)	8	Final embeddings were exported from the modified <code>finetune_final_004</code> run, initialized from <code>pretrain-epoch=016-val_loss=0.1065.ckpt</code> . Compared with the baseline, the modified variant produced more compact PCA embeddings and a comparable quick-probe profile, with mean accuracy 0.6868 vs. 0.6876 and mean ROC AUC 0.7730 vs. 0.7721.

Table 1: Baseline and modified autoencoder summary.

4.3 Part B: Autoencoder Architecture and Embedding Design

We followed the course-template autoencoder pipeline and used the learned latent representations as transferable features for downstream classification. The model takes a local 9×9 patch with 8 input channels, flattens it, and maps it into a low-dimensional latent space through a multilayer perceptron encoder; the decoder reconstructs the original patch from the latent code. For reproducibility, stage-specific training settings were stored in separate YAML files (`pretrain.yaml`, `finetune_cv.yaml`, and `finetune_final.yaml`). Our contribution in Part B was to modify the activation design, compare the resulting embeddings against the baseline template, and export the final latent features for downstream modeling.

4.3.1 Baseline and Modified Autoencoder

The baseline model is the template autoencoder with ReLU activations in the hidden layers of both encoder and decoder. Its encoder maps the flattened patch through two hidden layers of sizes 128 and 64 before projecting to the latent space, and the decoder mirrors this structure back to the original patch dimension.

Our modified model keeps the same encoder-decoder architecture and changes only the activation function from ReLU to LeakyReLU. Specifically, all hidden-layer activations were replaced by `LeakyReLU(negative_slope=0.1)` in both the encoder and decoder. This modification was motivated by the possibility of dead-ReLU behavior in the baseline network: once a unit is driven into a zero-gradient regime, it may stop contributing effectively to representation learning. LeakyReLU preserves a small gradient on the negative side and was therefore intended to yield a more stable latent representation while keeping the overall model capacity unchanged. By modifying only the activation function and leaving the layer widths and input/output design fixed, the comparison between the baseline and modified models isolates the effect of activation choice.

Table 1 summarizes the baseline and modified autoencoders used in our experiments. Both variants ultimately selected latent dimension 8, so the comparison was carried out under the same embedding size.

4.3.2 Embedding Evaluation and Selection Criteria

We evaluated the learned embeddings using three complementary criteria: qualitative visualization, latent-dimension comparison, and a quick downstream probe. First, we visualized the latent representations with PCA and t-SNE. Figures 2 and 3 compare the baseline and modified embeddings.

For the baseline model, the PCA projection shows substantial overlap between cloud and non-cloud pixels, whereas the t-SNE view reveals clearer local structure and partial class separation. This indicates that the baseline embedding already captures useful nonlinear information even though the first two principal components do not cleanly separate the two classes.

For the modified LeakyReLU model, the PCA embedding becomes visibly more compact, with fewer extreme outliers and a more concentrated central cloud of points. The class structure in the modified t-SNE visualization remains nonlinear and locally organized, while several regions exhibit slightly clearer cloud/no-cloud

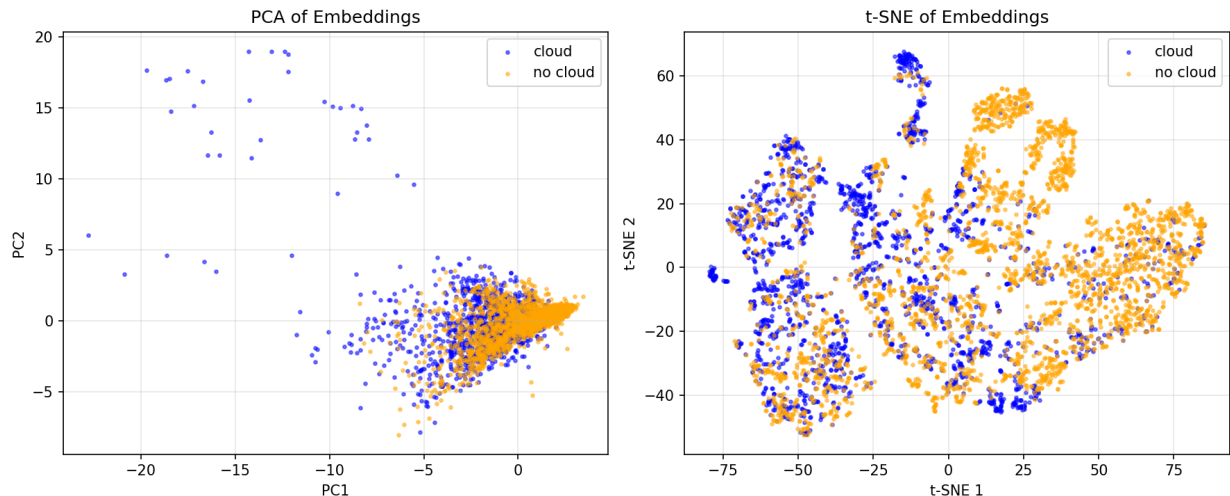


Figure 2: PCA and t-SNE visualizations of the baseline autoencoder embeddings. PCA shows strong overlap between cloud and non-cloud pixels, while t-SNE reveals clearer local clustering and partial separation.

grouping than in the baseline view. Overall, the modified model appears to preserve the useful local manifold structure of the baseline embedding while reducing some of the instability visible in the baseline PCA projection. This qualitative pattern is consistent with the intended effect of replacing ReLU by LeakyReLU.

Second, we compared candidate latent dimensions using a latent-dimension comparison script. Third, we used a quick probe to assess whether the learned representation was predictive under a lightweight downstream classifier. In practice, model selection was based on the joint evidence from these three diagnostics rather than on a single metric alone: latent-dimension comparison determined whether a compact representation was sufficient, the quick probe measured immediate downstream usefulness, and PCA/t-SNE served as a qualitative sanity check on embedding geometry.

4.3.3 Final Checkpoint and Latent Dimension

The final feature-extraction pipeline followed a pretraining–fine-tuning strategy. In the logged fine-tuning run, normalization statistics were loaded from `checkpoints/pretrain/norm_stats.npz`. For the baseline variant, the model was initialized from `pretrain-epoch=014-val_loss=0.1063.ckpt`; for the modified LeakyReLU variant, it was initialized from `pretrain-epoch=016-val_loss=0.1065.ckpt`. The run log further indicates that the final fine-tuning stage was built from 345,005 training patches and saved its output under `checkpoints/finetune/final`.

An important implementation detail is that the final fine-tuning log reports 92,488 trainable parameters out of 184,336 total parameters (50.17%). Moreover, the trainable parameter names shown in the log correspond to decoder layers. Accordingly, the log suggests a partially frozen setup in the final stage, likely preserving part of the learned encoder representation while allowing decoder-side adaptation during reconstruction-based fine-tuning. This interpretation is consistent with the transfer-learning goal of stabilizing the learned representation before downstream use.

For both the baseline and modified variants, the retained latent dimension used in downstream analysis is 8, as indicated by the feature columns `ae0`, `...`, `ae7`. The final exported embeddings for the modified model were taken from the modified `finetune_final_004` run, initialized from `pretrain-epoch=016-val_loss=0.1065.ckpt`. Relative to the baseline, the LeakyReLU model yielded comparable quick-probe mean accuracy (0.6868 vs. 0.6876), a slightly higher quick-probe mean ROC AUC (0.7730 vs. 0.7721), and a more compact PCA embedding at the same latent dimension. Taken together, these results suggest that the LeakyReLU variant is a competitive alternative to the baseline and is well suited for downstream experiments.

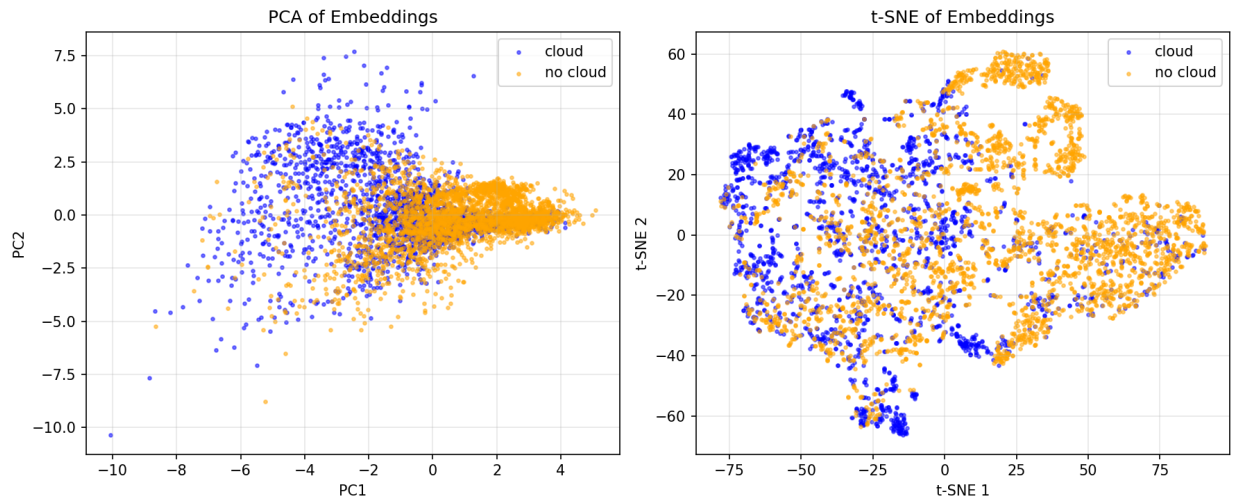


Figure 3: PCA and t-SNE visualizations of the modified LeakyReLU autoencoder embeddings. Compared with the baseline, the PCA representation is more compact and exhibits fewer extreme outliers, while the t-SNE view preserves meaningful local structure with slightly clearer class organization in several regions.

4.3.4 Extracted Latent Features for Downstream Classification

After training, we extracted latent features for the three labeled images and saved them as `image1_ae.csv`, ..., `image3_ae.csv`. Each file contains pixel coordinates (y, x) together with latent variables `ae0`, `ae1`, ..., `ae7`. This format preserves one-to-one alignment between each pixel and its embedding and therefore allows the latent representation to be merged directly with the original labeled data by spatial coordinates.

These exported embeddings form the interface between transfer learning and predictive modeling. In Part 3, they are used to construct both latent-only feature sets and combined raw-plus-latent feature sets by merging on (y, x) . This design allows us to test whether the learned representation contributes predictive information beyond the original handcrafted or expert-derived variables. Because the modified LeakyReLU embedding was more compact in PCA space and achieved a quantitative profile comparable to the baseline, with slightly higher mean ROC AUC in the quick-probe comparison, it was selected as the primary embedding source for the final downstream model comparison.

5 Modeling

5.1 Part A: Random Forest

5.1.1 Feature Sets and Model Setup

For the cloud classification task, we tested Random Forest with three feature settings: the original handcrafted features, the autoencoder latent features only, and the combination of both. The latent features were extracted from the fine-tuned encoder in the last part. For each labeled pixel, we used its corresponding 8-dimensional latent vector as the learned representation. The main goal was to check whether the transferred representation is useful for downstream cloud classification.

We chose Random Forest because it can model nonlinear relationships and feature interactions without strong distributional assumptions. In our implementation, the final model used 500 trees, maximum depth 16, minimum leaf size 10, `max_features="sqrt"`, and `class_weight="balanced_subsample"`. Since nearby pixels from the same image are highly dependent, we did not use random pixel-level splits. Instead, we used leave-one-image-out grouped cross-validation, so that each fold held out one full labeled image for testing.

5.1.2 Cross-Validation Results and Model Selection

We first compared the three feature settings under the same Random Forest model. Table 2 summarizes the overall leave-one-image-out results.

Table 2: Leave-one-image-out performance of the Random Forest classifier under three feature settings. Mean ROC-AUC was used as the main model selection criterion.

Feature set	Mean ROC-AUC	Mean Accuracy	Mean F1
handcrafted	0.8711	0.7745	0.7939
ae_only	0.9238	0.8334	0.7934
combined	0.9035	0.7771	0.7914

Among the three settings, the best model was the Random Forest using **ae_only** features. Its mean ROC-AUC across the three folds was 0.924, which was higher than both the handcrafted-only model and the combined model. Another useful point is that the **ae_only** model was also the most stable across folds. Its fold-wise AUC values were 0.924, 0.945, and 0.902, while the handcrafted and combined settings showed a larger drop on one held-out image. This suggests that the latent features learned by the autoencoder transfer better across different scenes.

5.1.3 ROC Curves and Fold-wise Performance

To examine the selected model in more detail, we plotted the ROC curves for the three leave-one-image-out folds.

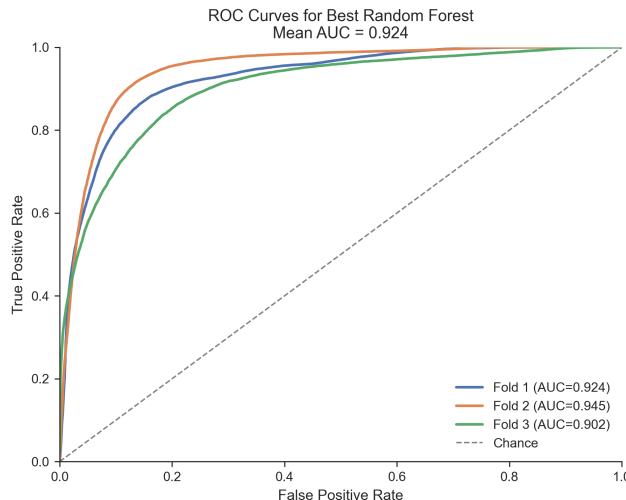


Figure 4: ROC curves for the selected Random Forest model using only autoencoder latent features. The model performs consistently across the three held-out labeled images, with mean AUC = 0.924.

Figure 10 shows that all three curves stay well above the diagonal baseline, and the fold-to-fold variation is not large. Fold 2 gives the strongest result, while Fold 3 is the weakest, but its AUC is still above 0.90. Overall, this plot suggests that the selected model has fairly consistent ranking performance on unseen labeled images.

5.1.4 Confusion Matrices

We also examined the confusion matrix for each fold to better understand the classification errors.

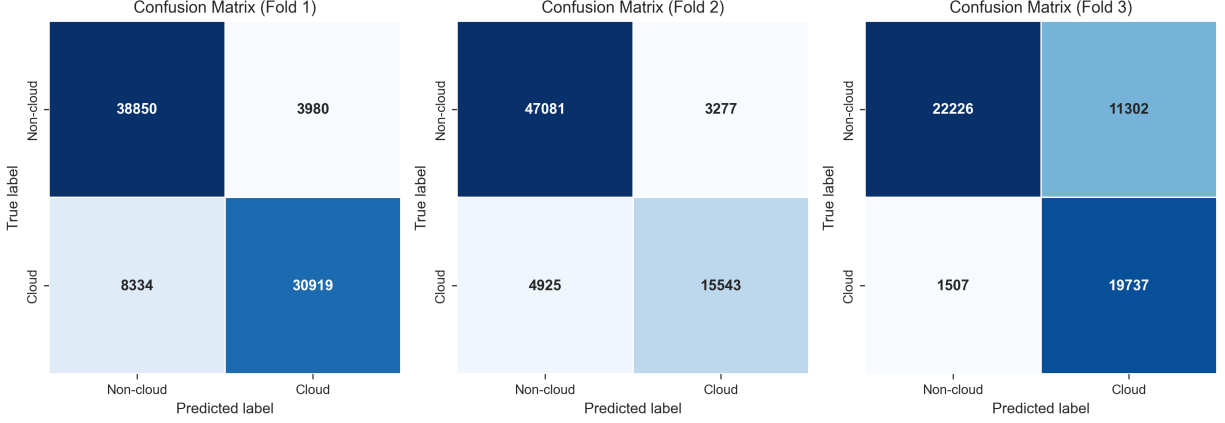


Figure 5: Confusion matrices for the three leave-one-image-out folds of the selected Random Forest model. Fold 3 shows more confusion between cloud and non-cloud pixels than the other two folds.

Figure 9 gives a more detailed view of the errors. In Fold 1 and Fold 2, the classifier performs strongly on both cloud and non-cloud pixels. Fold 3 is clearly more difficult. In that fold, the model still detects many cloud pixels correctly, but it produces more false positives for the non-cloud class. This pattern is consistent with the weaker ROC performance in Fold 3 and suggests that one labeled image is harder to separate than the other two.

5.1.5 Feature Importance

To understand which latent dimensions were most useful, we examined feature importance for the selected model using both MDI importance and permutation importance.

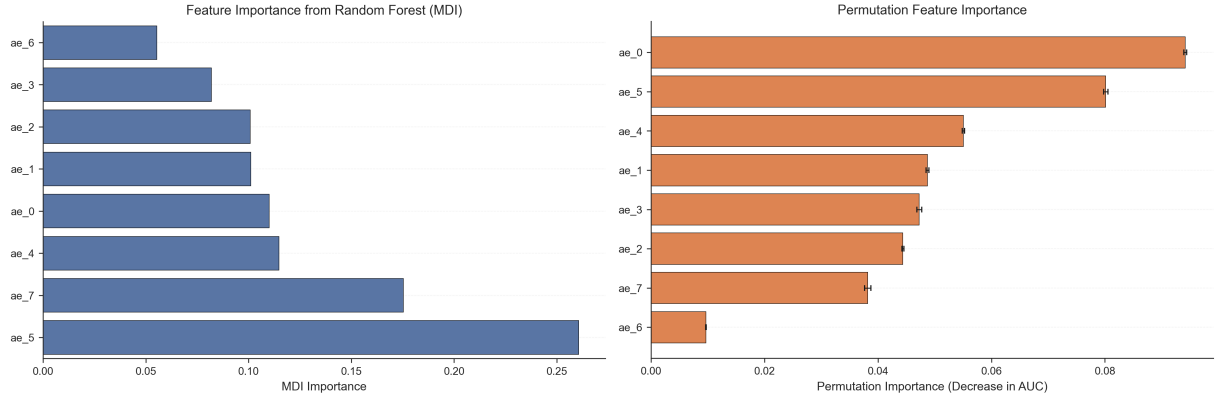


Figure 6: Feature importance for the selected **ae_only** Random Forest model. Left: MDI importance. Right: permutation importance based on decrease in ROC-AUC. The most influential latent dimensions are **ae_0** and **ae_5**, with **ae_4** and **ae_7** also contributing strongly.

Figure 6 shows that the two rankings are not identical, but they are broadly consistent. In particular, **ae_0** and **ae_5** appear among the most important latent features in both plots, while **ae_4** and **ae_7** also make relatively strong contributions. This suggests that the learned latent space is informative, but not all dimensions are equally important for cloud prediction.

5.1.6 Post-hoc Error Analysis

To better understand the model errors, we looked at the misclassification pattern across images and the prediction confidence distribution.

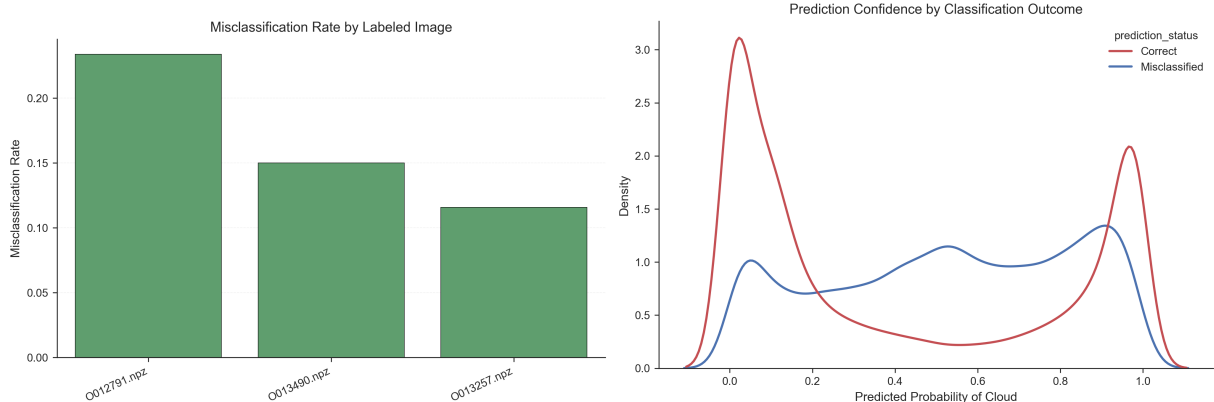


Figure 7: Post-hoc error analysis for the selected Random Forest model. Left: misclassification rate by labeled image. Right: predicted cloud probability for correct and incorrect classifications. Errors are more common in the more uncertain probability range.

Figure 7 shows that image `0012791.npz` has the highest misclassification rate, while `0013257.npz` has the lowest. This agrees with the cross-validation results and again suggests that some scenes are more difficult than others. The confidence plot shows another clear pattern: correct predictions are concentrated near probabilities close to 0 or 1, while misclassified points appear more often in the middle range. This means that many errors happen in less certain cases rather than in highly confident predictions.

5.1.7 Stability and Generalization Checks

Because the labeled dataset is small, we explicitly evaluated the stability of the selected Random Forest model. In our code, this was done in two ways. First, we repeated the grouped cross-validation under different random seeds and summarized the resulting AUC, F1, and accuracy. Since Random Forest includes randomness in tree construction, this test directly measures how sensitive the model is to that source of variation. Similar scores across seeds indicate that the model is stable and that the reported performance is not driven by one particular random run.

Second, we carried out a perturbation analysis by adding small Gaussian noise to the latent feature matrix and measuring how much the predictions changed. In particular, we recorded the prediction flip rate and the mean absolute probability shift. This analysis directly tests whether small changes in the learned representation lead to large changes in model output. A low flip rate and a small probability shift indicate that the classifier is robust to mild input perturbations.

Taken together, these analyses show that the selected model is not only accurate on the labeled images, but also stable under repeated fitting and small input perturbations.

5.1.8 Summary

Overall, the Random Forest results support the usefulness of the transfer learning stage. The best classifier used only the latent features from the fine-tuned autoencoder, and this model gave both the best mean ROC-AUC and the most stable performance across images. This suggests that the learned representation from the last part captures cloud-related structure more robustly than the handcrafted variables in this setting.

5.2 Part B: Logistic Regression

5.2.1 Why Logistic Regression

We used logistic regression as the first supervised baseline for cloud detection. This choice was motivated by three considerations. First, logistic regression is a standard and interpretable binary classifier whose probability output is directly aligned with the cloud/no-cloud labeling task. Second, it provides a clean way to compare feature sets without introducing substantial model complexity, which makes it especially useful for evaluating whether the learned latent representations from Part B contribute predictive signal beyond the

original handcrafted or expert-derived variables. Third, because logistic regression imposes a linear decision boundary in the supplied feature space, its performance serves as a useful diagnostic: strong results indicate that the available features already capture much of the relevant class structure, whereas weaker performance suggests that additional nonlinear modeling may still be beneficial.

5.2.2 Input Features and Data Construction

The logistic-regression experiments were built from the three labeled images used in the supervised portion of the lab. For each labeled pixel, we considered three feature constructions:

1. **Raw features only**, consisting of the original non-AE variables selected for downstream modeling;
2. **Latent features only**, consisting of the autoencoder embeddings exported as `ae0`, `...`, `ae7`;
3. **Raw + latent features**, formed by merging the two sources on pixel coordinates (y, x) .

The latent features were extracted from the transfer-learning pipeline described in the previous section. Because the exported embedding files preserve one-to-one alignment between each pixel and its coordinates, they can be merged directly with the labeled raw-feature table by (y, x) . This setup allows us to compare whether the autoencoder features are useful on their own and whether they add complementary information when combined with the original predictors.

5.2.3 Training/Test Design and Cross-Validation Strategy

To evaluate generalization to unseen images, we used an image-level leave-one-image-out design. In each fold, two labeled images were used for training and the remaining labeled image was held out for testing. This procedure was repeated three times so that each labeled image served once as the test image.

This design is preferable to a random pixel-level split because nearby pixels within the same image are highly correlated. A random pixel split would therefore lead to overly optimistic performance estimates by allowing nearly identical local structures to appear in both training and test sets. By holding out an entire image at a time, the evaluation better reflects the intended deployment setting in which the model is applied to a new image rather than to neighboring pixels from an already-seen image.

5.2.4 Logistic Regression Specification and Assumption Checks

For each feature set, logistic regression models the cloud probability through

$$\Pr(Y = 1 \mid X = x) = \frac{1}{1 + \exp(-(\beta_0 + x^\top \beta))}.$$

Thus, the classifier assumes that the log-odds of cloud presence can be approximated by a linear function of the supplied predictors. We evaluated each model using ROC AUC, balanced accuracy, and F1 score on the held-out image in each fold.

This model family is most appropriate when the supplied feature space is already reasonably informative. In our setting, this assumption appears more plausible for the raw and raw-plus-latent feature sets than for the latent-only feature set. In particular, the latent-only models achieve moderate ROC AUC but substantially weaker threshold-based metrics, suggesting that the learned embeddings carry useful ranking information while still leaving some class overlap or threshold-calibration issues unresolved. Conversely, the raw-feature models achieve consistently stronger balanced accuracy and F1, indicating that a linear separator in the original feature space is already competitive for this task.

5.2.5 Comparison Across Feature Sets

Table 3 summarizes the mean cross-validated performance across the three held-out images. For the baseline embeddings, the raw-only logistic model achieved the strongest balanced accuracy and F1 score, while the raw-plus-latent model achieved the highest mean ROC AUC. The latent-only model was clearly weaker than the other two feature sets in threshold-based metrics.

The same overall pattern remained after replacing ReLU by LeakyReLU in the autoencoder. Raw-only logistic regression remained the strongest model in balanced accuracy and F1, whereas raw-plus-latent again gave the highest mean ROC AUC. This suggests that the latent features improved ranking performance more than hard classification performance at the default decision threshold. In other words, the autoencoder representation appears to help the model order pixels by cloud likelihood, but that improvement does not

Embedding source	Feature set	AUC	Balanced Acc.	F1
Baseline	Raw	0.813	0.791	0.719
Baseline	Latent	0.746	0.660	0.565
Baseline	Raw + Latent	0.824	0.766	0.687
Modified	Raw	0.813	0.791	0.719
Modified	Latent	0.753	0.665	0.573
Modified	Raw + Latent	0.825	0.765	0.687

Table 3: Mean leave-one-image-out logistic-regression performance across feature sets. The raw-only model achieves the best threshold-based metrics, while the raw-plus-latent model achieves the highest mean AUC.

automatically translate into better thresholded predictions than the strong raw baseline.

At the fold level, the held-out image results reveal that image2 is consistently the easiest of the three labeled images: both raw and raw-plus-latent models achieve very high AUC, balanced accuracy, and F1 on this fold. In contrast, image1 and image3 are substantially more challenging. This fold-level heterogeneity suggests that cloud structure and background conditions vary meaningfully across images, reinforcing the importance of image-level validation.

5.2.6 Baseline vs. Modified Latent Features

The activation change in the transfer-learning stage led to modest but consistent improvements in the latent-only logistic results. Relative to the baseline latent features, the modified LeakyReLU embeddings improved mean AUC from 0.746 to 0.753, mean balanced accuracy from 0.660 to 0.665, and mean F1 from 0.565 to 0.573. These gains are small in magnitude, but they are directionally consistent across the aggregate summary and support the claim that the modified embedding is at least slightly more useful as a standalone representation.

The effect of the modified embeddings is more limited once raw features are added. For the raw-plus-latent model, the modified embeddings produce a very small increase in mean AUC (0.824 to 0.825), while balanced accuracy and F1 remain essentially unchanged. This indicates that most of the discriminative power in the combined model still comes from the raw features, with the latent representation contributing only a modest incremental benefit. Taken together, these results are consistent with the transfer-learning findings from Part B: the LeakyReLU modification improves embedding quality, but the downstream gain is most visible when the model relies directly on the latent features rather than when those features are appended to an already-strong raw baseline.

5.2.7 Coefficient Importance and Misclassification Analysis

Although logistic regression is attractive partly because of its interpretability, the current experiment outputs mainly provide performance metrics and confusion counts rather than a full coefficient summary. As a result, our present analysis emphasizes error patterns across folds.

Several patterns emerge from the confusion matrices. First, the latent-only models tend to make many more false positives than the raw-only models, especially on image1, where the latent representation identifies many cloud pixels correctly but also overpredicts cloud in non-cloud regions. This behavior is consistent with the latent-only model achieving reasonable AUC but weaker balanced accuracy and F1. Second, the raw-only model is much more conservative on image1, producing far fewer false positives but also many fewer true positives than the latent-only model. Third, on image2 the raw and raw-plus-latent models both perform extremely well, indicating that this image is relatively easy for linear decision boundaries in either feature space.

For image3, the latent-only models remain the weakest, with lower balanced accuracy and F1 than the raw-based alternatives. The raw-plus-latent model improves the ranking performance relative to raw-only, but this does not translate into uniformly better threshold-based predictions. Overall, the misclassification patterns suggest that latent features capture useful cloud-like structure but are not yet as sharply calibrated as the raw features under a linear classifier.

A fuller interpretability analysis should include a ranked coefficient table or coefficient-magnitude plot for

standardized predictors. That would allow us to identify which raw variables and which latent coordinates most strongly drive the fitted cloud probabilities.

5.2.8 Role in the Final Model Comparison

Within the broader predictive-modeling section, logistic regression serves as the most interpretable baseline. Its main value is not only as a standalone classifier, but also as a diagnostic tool for understanding what the engineered and learned feature spaces already provide before moving to more flexible nonlinear models.

The logistic results suggest three main conclusions. First, the original raw features are already very strong under a linear model, especially in terms of balanced accuracy and F1. Second, latent features alone are informative but not yet competitive with the raw baseline. Third, combining raw and latent features slightly improves AUC but does not clearly improve threshold-based performance. Therefore, logistic regression establishes a useful benchmark: any more complex model should be judged against a raw-feature logistic baseline that is already quite effective, rather than against a weak or uninformative starting point.

5.3 Part C: Linear Discriminant Analysis

5.3.1 Model Setup and Feature Sets

In this part, Linear Discriminant Analysis (LDA) is used for cloud detection. LDA is a probabilistic classifier that is suitable when the two classes can be separated by linear combinations of predictors. LDA assumes that, conditional on the class label, the predictor vector follows a multivariate Gaussian distribution with class-specific mean vectors and a common covariance matrix shared across classes. Under these assumptions, the resulting classifier produces a linear decision boundary in feature space.

The LDA experiments were built from the same three labeled images used in the supervised modeling stage. For each labeled pixel, we considered three feature constructions: handcrafted features only, Latent features only, and handcrafted + latent features.

5.3.2 Cross-Validation Results and Model Selection

We used the same leave-one-image-out cross-validation design as in the other supervised models. Table 4 summarizes the mean leave-one-image-out performance of LDA under the three feature settings.

Feature Set	Mean AUC	Mean Accuracy	Mean F1 Score
handcrafted	0.8242	0.6426	0.8150
ae_only	0.9100	0.6677	0.7878
combined	0.8728	0.6441	0.8085

Table 4: Mean leave-one-image-out performance of LDA under different feature sets.

The best feature set by mean ROC-AUC is ae_only, with a mean ROC-AUC of 0.9100. The combined feature set improves over ‘handcrafted’ in AUC, but it does not outperform ae_only. handcrafted achieves the highest mean accuracy (0.8150), but its AUC is notably lower than ae_only. It shows the autoencoder representation captures information that improves ranking and class separation, even when threshold-based accuracy is not maximized.

5.3.3 ROC Curves and Fold-wise Performance

In this section, we examined how LDA performed on each held-out image separately.

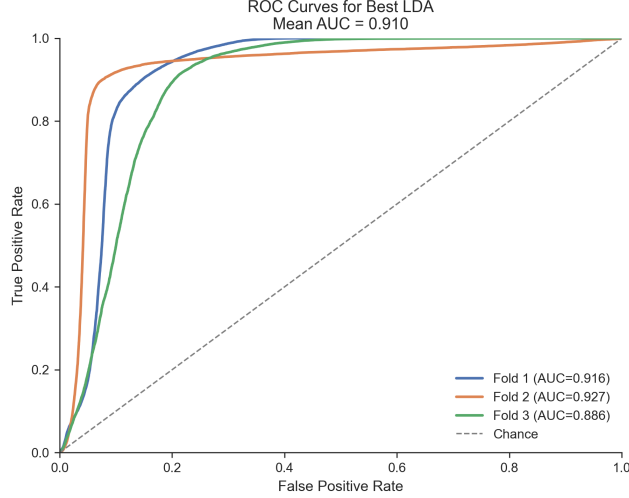


Figure 8: Fold-wise LDA performance across the three leave-one-image-out splits.

The best mean AUC is approximately 0.91 from `ae_only`. There is still some variation across folds. The fold-specific AUC values are approximately 0.916, 0.928, and 0.886. The second fold achieves the best performance, while the third fold is weaker, but the ROC curve remains above the chance baseline.

5.3.4 Confusion Matrices

To better understand the classification behavior of LDA, we examined the confusion matrix for each leave-one-image-out fold.

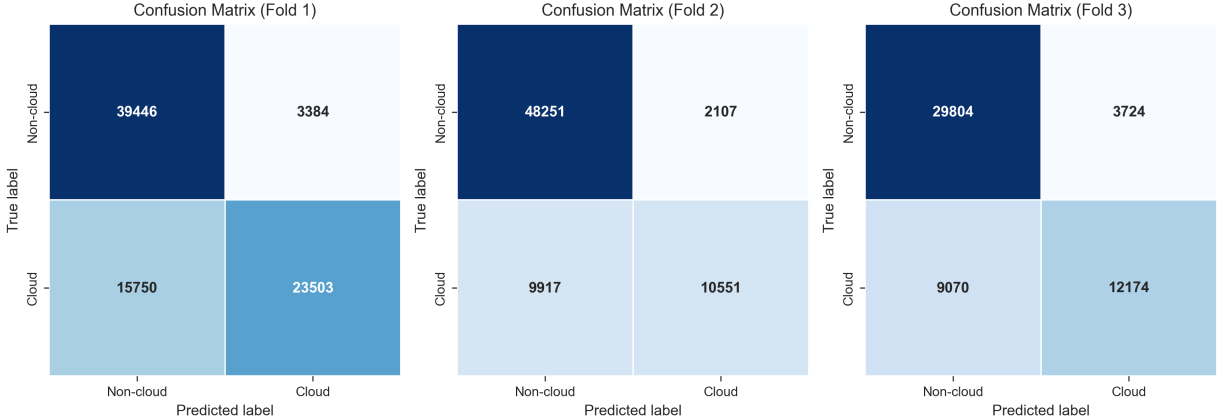


Figure 9: Confusion matrices for the three leave-one-image-out folds under the selected LDA feature setting.

In Fold 1, the model achieved 39,446 true negatives and 23,503 true positives, but still produced 15,750 false negatives, indicating that some cloudy pixels were missed. Fold 2 was the most conservative, with only 2,107 false positives but 9,917 false negatives, which explains its high precision (0.834) but relatively low recall (0.515). Fold 3 showed a more balanced but still imperfect pattern, with 29,804 true negatives, 12,174 true positives, 3,724 false positives, and 9,070 false negatives.

5.3.5 Interpretation of the Discriminant Directions

One advantage of LDA is that it remains relatively interpretable. Because the classifier is based on a linear discriminant rule, we can inspect the estimated discriminant coefficients to understand which predictors contribute most strongly to the separation between cloud and non-cloud pixels.

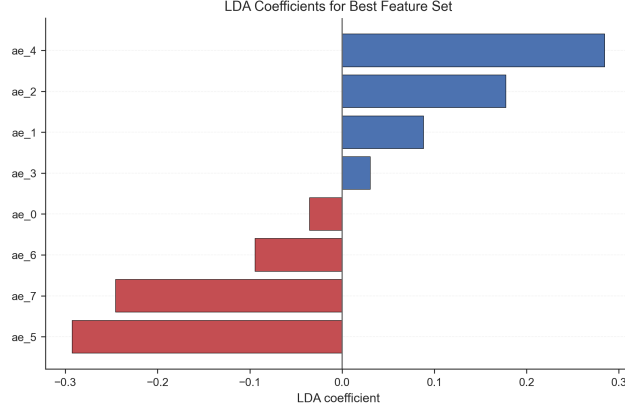


Figure 10: Estimated LDA discriminant coefficients under the selected feature setting.

The coefficient plot for the best LDA model shows the relative contribution of each autoencoder latent dimension to the linear decision boundary. Positive coefficients indicate that larger values of the corresponding latent feature push predictions toward the cloud class, while negative coefficients favor the non-cloud class. In our results, ae_4 and ae_2 had the largest positive coefficients, whereas ae_5 and ae_7 had the largest negative coefficients, suggesting that these latent directions are most influential for discrimination.

6 Bibliography

Shi, T., Yu, B., Clothiaux, E. E., and Braverman, A. J. (2008). Daytime Arctic Cloud Detection Based on Multi-Angle Satellite Data With Case Studies. *Journal of the American Statistical Association*, 103(481), 584–597.

A Academic honesty

A.1 Statement

We certify that this report and the associated code were completed in accordance with the course academic honesty policy. All work submitted reflects our group’s own understanding, analysis, implementation, and writing. Any external sources, including research papers and software tools, have been appropriately acknowledged. We did not copy solutions from other students or unauthorized sources, and we did not submit work that was not meaningfully produced and verified by our group.

A.2 LLM Usage

Coding

We used large language model (LLM) tools in a limited supporting role during the coding process. Specifically, we used AI assistance to help generate figures, troubleshoot errors, and suggest revisions to parts of the code. All code was reviewed, tested, and modified by us as needed before being included in the final submission. We take full responsibility for the correctness, design, and interpretation of the final code and results.

Writing

We used LLM tools to help improve the clarity, grammar, and overall polish of the writing in this report. The underlying ideas, experimental design, analysis, and conclusions were developed by our group. AI assistance was used only for language refinement rather than for generating the substantive academic content of the report.